DEEC
DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA E DE COMPUTADORES
TÉCNICO LISBOA

Licenciatura em Engenharia Electrotécnica e de Computadores (LEEC)

Algoritmos e Estruturas de Dados - 2026/2027
Aula de Laboratório #04 - 19 a 23 de Outubro de 2025
Ordenação

Objectivos

Este laboratório estrutura-se em duas partes, cada uma com um objetivo específico. Na primeira parte, como em laboratórios anteriores, o objetivo é discutir e comparar, mediante métricas simples de complexidade, soluções para um problema muito simples e desenvolver uma solução computacional para o mesmo. Não é fornecido qualquer código que se espera ser desenvolvido na aula, coletivamente.
Na segunda parte focamos a nossa atenção no tema da ordenação de dados. Como exemplo de trabalho, voltamos ao problema de contar palavras distintas num ficheiro de texto, problema que foi usado no Laboratório #03.

1 Primeira Parte

Dado um vector com `N` inteiros e um valor inteiro alvo, `target`, pretende-se implementar uma função que encontre todos os pares únicos de elementos nesse vector cuja diferença entre o maior e o menor deles seja exactamente igual a `target`.
A assinatura da função deverá ser:

```
void find_diff_tuples(int* arr, int N, int target, int** tuples, int* t_count);
```

onde `arr` é o vector de entrada, `N` é o seu tamanho, `target` é a diferença alvo, `tuples` é um duplo ponteiro que será usado para armazenar os pares e `t_count` será usado para armazenar a contagem total de pares que satisfazem a condição.

a) Discuta possíveis soluções algoritmicas para o problema comparando-as quando à sua complexidade computacional e de memória.

b) Escreva uma implementação do algoritmo proposto. Deverá assumir que a variável `tuples` não está alocada. Assim, a sua função deverá alocá-la na dimensão adequada (ver nota abaixo) e preenche-la de forma a que no final da execução cotenha todos os pares de elementos que satisfazem a condição indicada.

   Nota: Pode assumir que o vector não possui elementos repetidos.

c) Proponha uma expressão analítica para a complexidade temporal da sua implementação, que deverá estar de acordo com a implementação apresentada em em a) e implementada em b).

2 Segunda Parte

Neste trabalho utilizaremos dois módulos em `C` que no seu conjunto permitirão ilustrar o problema da ordenação. Os módulos, denominados `sortelems.c` e `elem.c` contêm código incompleto em que a organização dos dados é feita numa tabela como discutido no 1° problema da segunda parte do Laboratório #03. A diferença é que agora em vez de usarmos palavras (i.e. strings) como nesse laboratório, aqui usaremos apenas inteiros não negativos. Cada elemento da tabela aponta para uma estrutura (tipo

`IntO`), que contém dois inteiros, um para o número e outro para o número de ocorrências do número. Os argumentos deste código são dois. O primeiro argumento é um ficheiro de dados em que na 1ª linha se indicam quantos números N estão no ficheiros seguindo-se N números inteiros. Em anexo foram disponibilizados vários ficheiros deste tipo de dimensões sucessivamente maiores.

O processamento a fazer passa por não apenas a contabilização do número de inteiros diferentes mas também a sua ordenação, que poderá ser segundo um dos seguintes critérios:

A. magnitude ou valor dos números

B. número de ocorrências de cada número.

O critério de ordenação é especificado pelo utilizador na linha de comando. Para ordenação por ordem por magnitude deverá ser usado o comando

`./sortelems ficheiro.txt -magnitude{-a|-d}`,

onde os parâmetros `a` e `d` indicam, respectivamente, ordenação ascendente e ordenação descendente. Para ordenação por número de ocorrências deverá ser usado o comando

`./sortelems ficheiro.txt -ocorrencias {-a|-d}`.

Por omissão, a ordenação deverá ser por magnitude e ascendente, usando o algoritmo implementado na função `sort()` dada.

Estude o código fornecido, para compreender como está organizado, qual o algoritmo subjacente e de que forma os dados são guardados. Verifique os cabeçalhos das funções de interface, e note que o último argumento da função `sort()` é um ponteiro para uma função:

```
void sort(Item a[], int l, int r, int (*compare) (Item, Item));
```

Use o comando `make` e o ficheiro `Makefile`, com as alterações que entender necessárias, para compilar o programa e fazer testes. Como se disse acima, a pasta `tests` tem alguns exemplos de dados no formato indicado, mas pode obviamente usar outros de maior dimensão.

1. Identifique o algoritmo de ordenação implementado pela função `sort()` fornecida, e implemente a sua chamada no programa principal.

2. Escreva as restantes funções de comparação para implementar todos os critérios de ordenação. Liberte também a memória alocada.

3. Teste o programa com os diferentes critérios de ordenação e diferentes ficheiros de entrada.

4. Altere o programa de modo a avaliar as operações de acesso efectuadas. Obtenha dados experimentais usando ficheiros de teste, e crie uma tabela com regressão mostrando que o algoritmo é $\mathcal{O}\left(n^2\right)$.

5. Crie uma versão `qsortelems.c`, em que o conteúdo da função `sort()` é substituído por uma implementação sua do algoritmo Quicksort (pode ver o código da mesma nos acetatos da disciplina).

6. Obtenha dados experimentais de teste que lhe permitam comparações com o programa anterior e tirar conclusões sobre a complexidade do Quicksort.